

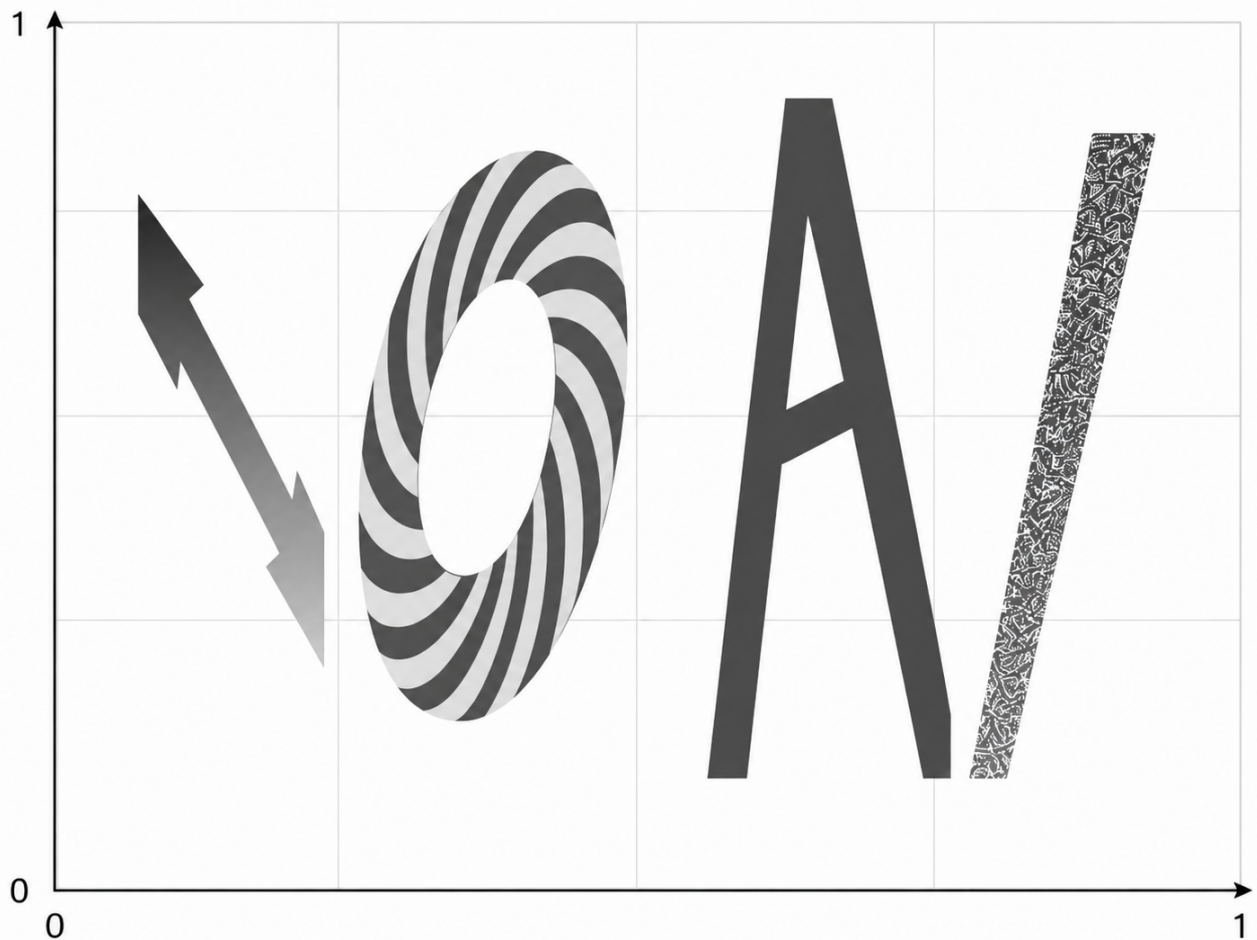
Trường IOAI

- **Giới hạn thời gian:** 5 phút
- **Dung lượng lưu trữ:** 5 GB
- **Kích thước lời giải:** tổng cộng `solution.ipynb`, `custom_model.py` ≤ 1 MB
- **Mô hình được huấn luyện trước:** không — huấn luyện từ đầu, không có internet khi chấm
- **Điểm Baseline:** 31.2187

Nhiệm vụ

Thị trưởng Astana muốn trang trí thành phố bằng các logo IOAI cách điệu. Là một nhà thống kê, ông xem mọi thứ—bao gồm cả logo—như một hàm không gian $F(x, y, \overline{W})$, trong đó $x, y \in [0, 1]$ biểu diễn các tọa độ trên mặt phẳng 2D và $\overline{W}$ là một tập hợp các tham số ẩn xác định những thuộc tính phong cách như màu sắc và góc nghiêng của các chữ cái.

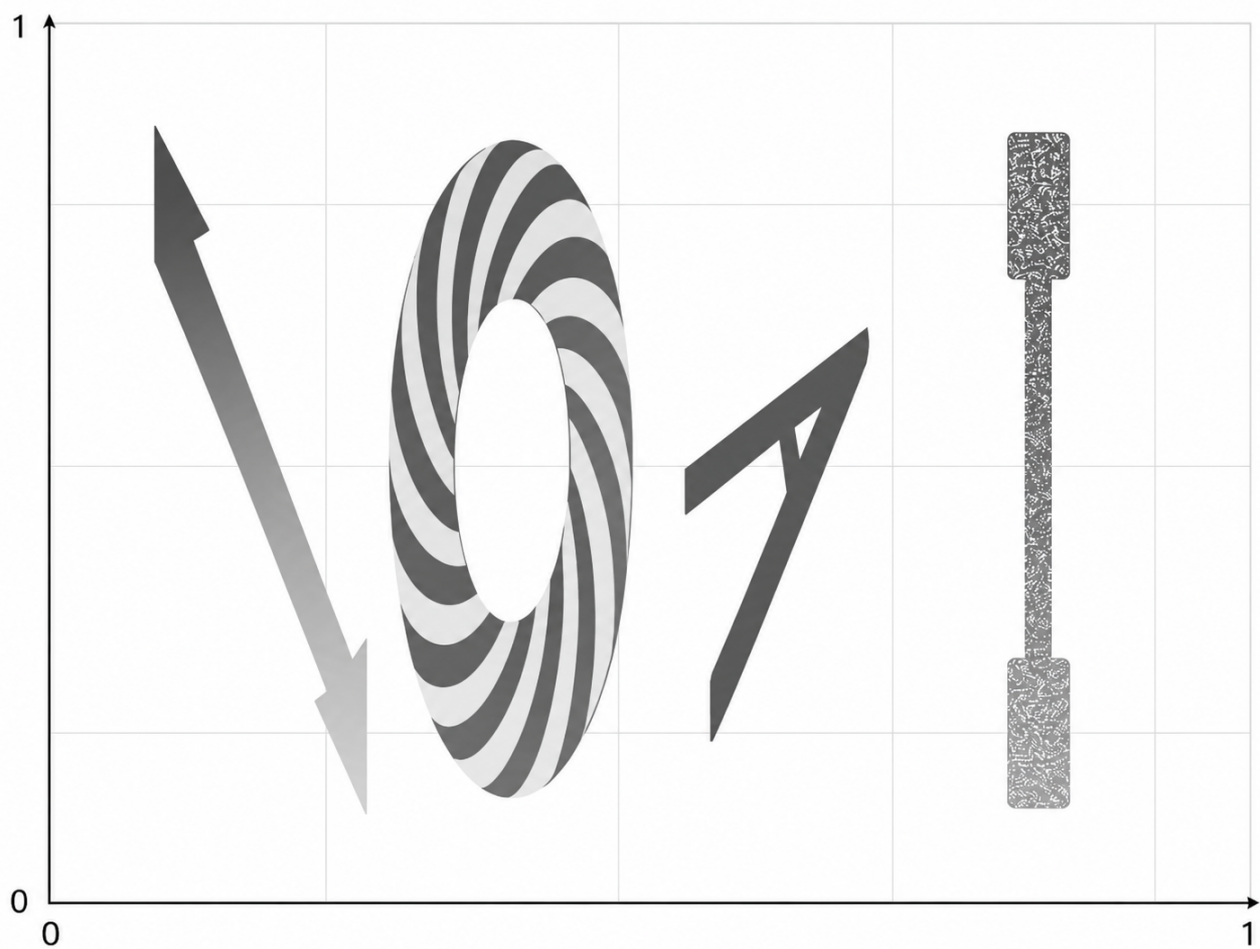
Vì F quá phức tạp để biểu diễn dưới dạng một phương trình toán học tường minh, nhiệm vụ của bạn là huấn luyện một mạng nơ-ron để xấp xỉ hàm này. Mạng sẽ xuất ra một giá trị **trường IOAI** cho bất kỳ cặp tọa độ (x, y) nào, tạo ra một hình ảnh heatmap hoàn chỉnh của logo trên toàn mặt phẳng. Dưới đây là một ví dụ về hình ảnh heatmap của F với một số tham số ẩn cụ thể $\overline{W}$.

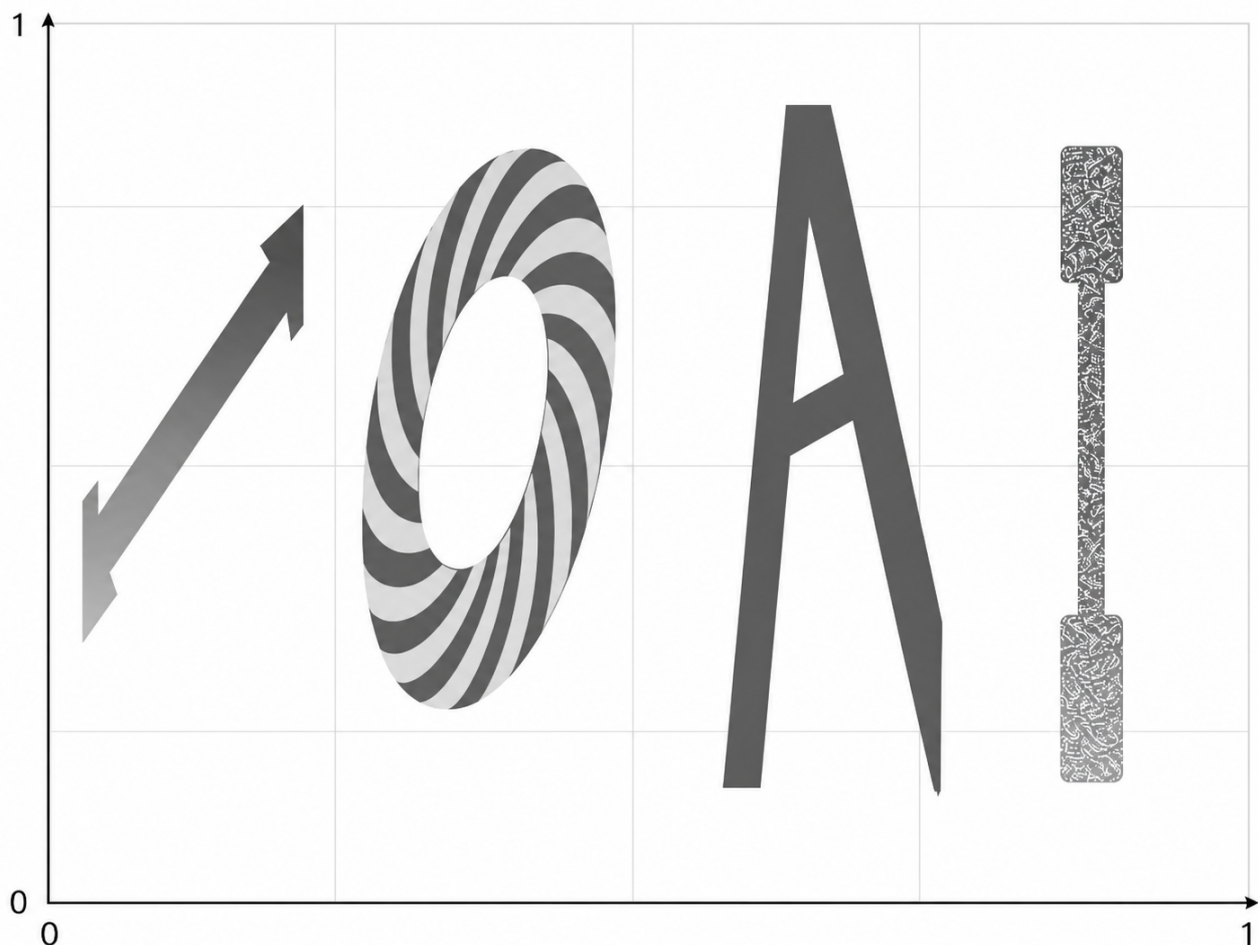


Trường IOAI gồm những gì? Bốn chữ cái và phần nền.

- Các giá trị bên trong chữ cái $\mathbb{I}$ đầu tiên rất lớn ($1e+10$ trở lên), với một gradient tuyến tính
- Các giá trị trong chữ cái $\circ$ thể hiện một mẫu hình xoắn ốc
- Giá trị bên trong chữ cái $\mathbb{A}$ luôn là -1
- Các giá trị bên trong chữ cái $\mathbb{I}$ cuối cùng phải là các giá trị ngẫu nhiên thuộc khoảng $[-2026, 2026]$, ngay cả khi được đánh giá tại cùng một điểm hai lần
- Bên ngoài các chữ cái, giá trị luôn bằng không

Hàm có các tham số ẩn $\overline{W}$, ảnh hưởng đến tỷ lệ và độ nghiêng của các chữ cái, cùng với khoảng giá trị bên trong chữ cái $\mathbb{I}$ đầu tiên. Tuy nhiên, các chữ cái sẽ không giao nhau. Dưới đây là một vài ví dụ minh họa về hình dạng của trường IOAI với các giá trị $\overline{W}$ khác nhau:





Những gì bạn được cung cấp:

Bài toán này KHÔNG chứa dataset. Thay vào đó, bạn được cung cấp hàm sinh được cấu hình bởi tệp config JSON tại `data/train_config/field_config.json`.

Config kiểm thử được ẩn, nhưng có bản chất tương tự. Nhiệm vụ của bạn là khớp với hàm sinh đã cho bằng cách sử dụng lượng dữ liệu tùy ý theo ý bạn. Các phân phối "train" và "test" của bạn được tạo từ cùng một hàm sinh — bạn chỉ không biết mình sẽ được đánh giá trên những điểm (x_i, y_i) nào.

Bài nộp của bạn phải bao gồm:

- lớp model huấn luyện được lưu dưới dạng `custom_model.py`. Model này phải kế thừa từ lớp `torch.nn.Module` và chỉ sử dụng các import `torch`. Tệp phải chứa lớp `CustomModel` được sử dụng trong notebook `solution.ipynb`.
- notebook `solution.ipynb`, sẽ tạo ra các trọng số `model.pt`

Chấm điểm

Đối với mỗi vùng, điểm tối thiểu là 0 và điểm tối đa là 1. Điểm cuối cùng được lấy trung bình trên cả năm vùng (bốn vùng tương ứng với từng chữ cái và phần nền) rồi nhân với 100. Có một **mức**

phạt theo số lượng tham số:

Nếu model của bạn có nhiều hơn 20260 tham số, điểm sẽ bị giảm một nửa.

Số lượng tham số được đo bằng `sum(p.numel() for p in model.parameters())`. Chúng tôi cũng yêu cầu model của bạn hoạt động ở chế độ ngẫu nhiên, với `nn.Dropout` của PyTorch là một phần của model.

Đối với các vùng tiêu chuẩn

Đối với mỗi vùng R (chữ cái $\mathbb{I}$ đầu tiên, O , A , $Background$), chúng tôi đánh giá model trên $N_R = 512$ điểm kiểm thử (x_i, y_i) với các giá trị thực v_i và các giá trị dự đoán $\hat{v}_i$. Chúng tôi sử dụng Sai số Tuyệt đối Trung bình (Mean Absolute Error, MAE) đã chuẩn hóa làm metric chính. MAE được định nghĩa như sau:

$$MAE(R) = \frac{1}{N_R} \sum_{i=1}^{N_R} |\hat{v}_i - v_i|.$$

Và việc chuẩn hóa được thực hiện như sau

$$score(R) = 1 - \min\left(\frac{MAE(R)}{s_R}, 1\right),$$

trong đó $s_R > 0$ là một hằng số tỷ lệ.

Đối với vùng của chữ cái $\mathbb{I}$ cuối cùng

Trong vùng này, **dropout được bật trong quá trình đánh giá**. Đối với mỗi điểm kiểm thử j :

1. Chúng tôi chạy model $K = 10$ lần để thu được $\hat{v}_{j,1}, \dots, \hat{v}_{j,K}$.
2. Nếu bất kỳ đầu ra nào nằm ngoài khoảng $[-2026, 2026]$, thì $pointScore(j) = 0$.
3. Nếu không, tính độ lệch chuẩn σ_j của K đầu ra và chuyển đổi nó thành điểm số:

$$pointScore(j) = \min\left(\frac{\sigma_j}{s_E}, 1\right),$$

trong đó $s_E > 0$ là một hằng số tỷ lệ cố định.

Điểm của vùng là giá trị trung bình trên tất cả các điểm trong vùng:

$$score(I_{last}) = \frac{1}{N_E} \sum_{j=1}^{N_E} pointScore(j),$$

trong đó $N_E = K * N_R$.

Nói một cách đơn giản, mức độ đa dạng càng cao thì điểm của bạn cho vùng này càng lớn. **Bạn không thể sử dụng tính ngẫu nhiên ở dạng thuần túy, bao gồm các hàm `rand*` và `_uniform`**

của PyTorch; tính ngẫu nhiên phải đến từ quá trình suy luận với dropout được bật.

Cách nộp bài

1. Mở `solution.ipynb` và chạy tất cả các cell.
2. Cải thiện model `CustomModel` trong `custom_model.py`
3. Đảm bảo rằng cell cuối cùng của bạn lưu model vào tệp `model.pt`.
4. Trong tab Git của JupyterLab, stage, ghi chú và commit `solution.ipynb` cùng `custom_model.py`, sau đó push.
5. Quay lại trang Cuộc thi và nhấp vào **Nộp bài**. Ghi chú nộp bài phải giống với ghi chú ở bước trước.